

4.4节曾仿效古希腊英雄忒修斯，以栈等基本数据结构模拟线绳和粉笔，展示了试探回溯策略的应用技巧。实际上，这一技巧可进一步推广至更为一般性的场合，包括可以图结构描述的应用问题，从而导出一系列对应的图算法。

忒修斯取得成功的关键在于，借助线绳掌握迷宫内各通道之间的联接关系。在很多应用中，能否有效描述和利用这类信息，同样至关重要。一般地，这类信息往往可表述为定义于一组对象之间的二元关系，比如城市交通图中，联接于各公交站之间的街道，或者互联网中，联接于IP节点之间的路由，等等。尽管在某种程度上，第5章所介绍的树结构也可用以表示这种二元关系，但仅限于父、子节点之间。相互之间均可能存在二元关系的一组对象，从数据结构的角度的分类，属于非线性结构（non-linear structure）。此类一般性的二元关系，属于图论（Graph Theory）的研究范畴。从算法的角度对此类结构的处理策略，与上一章相仿，也是通过遍历将其转化为半线性结构，进而借助树结构已有的处理方法和技巧，最终解决问题。

以下首先简要介绍图的基本概念和术语，已有相关基础的读者可直接跳过。接下来，介绍如何实现作为抽象数据类型的图结构，主要讨论邻接矩阵和邻接表两种实现方式。然后，从遍历的角度介绍将图转化为树的典型方法，包括广度优先搜索和深度优先搜索。进而，分别以拓扑排序和双连通域分解为例，介绍利用基本数据结构并基于遍历模式，设计图算法的主要方法。最后，从“数据结构决定遍历次序”的观点出发，将所有遍历算法概括并统一为最佳优先遍历这一模式。如此，我们不仅能够更加准确和深刻地理解不同图算法之间的共性与联系，更可以学会通过选择和改进数据结构，高效地设计并实现各种图算法——这也是本章的重点与精髓。

§ 6.1 概述

■ 图

图结构是描述和解决实际应用问题的一种基本而有力的工具。所谓的图（graph），可定义为 $G = (V, E)$ 。其中，集合 V 中的元素称作顶点（vertex）；集合 E 中的元素分别对应于 V 中的某一对顶点 (u, v) ，表示它们之间存在某种关系，故亦称作边（edge）^①。一种直观显示图结构的方法是，用小圆圈或小方块代表顶点，用联接于其间的直线段或曲线弧表示对应的边。

从计算的需求出发，我们约定 V 和 E 均为有限集，通常将其规模分别记 $n = |V|$ 和 $e = |E|$ 。

■ 无向图、有向图及混合图

若边 (u, v) 所对应顶点 u 和 v 的次序无所谓，则称作无向边（undirected edge），例如表示同学关系的边。反之若 u 和 v 不对等，则称 (u, v) 为有向边（directed edge），例如描述企业与银行之间的借贷关系，或者程序之间的相互调用关系的边。

^① 在某些文献中，顶点也称作节点（node），边亦称作弧（arc），本章则统一称作顶点和边。

如此, 无向边 (u, v) 也可记作 (v, u) , 而有向的 (u, v) 和 (v, u) 则不可混淆。这里约定, 有向边 (u, v) 从 u 指向 v , 其中 u 称作该边的起点 (origin) 或尾顶点 (tail), 而 v 称作该边的终点 (destination) 或头顶点 (head)。

若 E 中各边均无方向, 则 G 称作无向图 (undirected graph, 简称undigraph)。例如在描述影视演员相互合作关系的图 G 中, 若演员 u 和 v 若曾经共同出演过至少一部影片, 则在他 (她) 们之间引入一条边 (u, v) 。反之, 若 E 中只含有向边, 则 G 称作有向图 (directed graph, 简称digraph)。例如在C++类的派生关系图中, 从顶点 u 指向顶点 v 的有向边, 意味着类 u 派生自类 v 。特别地, 若 E 同时包含无向边和有向边, 则 G 称作混合图 (mixed graph)。例如在北京市内交通图中, 有些道路是双行的, 另一些是单行的, 对应地可分别描述为无向边和有向边。

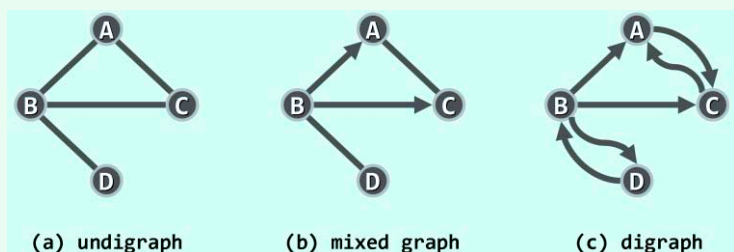


图6.1 (a)无向图、(b)混合图和(c)有向图

相对而言, 有向图的通用性更强, 因为无向图和混合图都可转化为有向图——如图6.1所示, 每条无向边 (u, v) 都可等效地替换为对称的一对有向边 (u, v) 和 (v, u) 。因此, 本章将主要针对有向图, 介绍图结构及其算法的具体实现。

度

对于任何边 $e = (u, v)$, 称顶点 u 和 v 彼此邻接 (adjacent), 互为邻居; 而它们都与边 e 彼此关联 (incident)。在无向图中, 与顶点 v 关联的边数, 称作 v 的度数 (degree), 记作 $\deg(v)$ 。以图6.1(a)为例, 顶点 $\{A, B, C, D\}$ 的度数为 $\{2, 3, 2, 1\}$ 。

对于有向边 $e = (u, v)$, e 称作 u 的出边 (outgoing edge)、 v 的入边 (incoming edge)。 v 的出边总数称作其出度 (out-degree), 记作 $\text{outdeg}(v)$; 入边总数称作其入度 (in-degree), 记作 $\text{indeg}(v)$ 。在图6.1(c)中, 各顶点的出度为 $\{1, 3, 1, 1\}$, 入度为 $\{2, 1, 2, 1\}$ 。

简单图

联接于同一顶点之间的边, 称作自环 (self-loop)。在某些特定的应用中, 这类边可能的确具有意义——比如在城市交通图中, 沿着某条街道, 有可能不需经过任何交叉路口即可直接返回原处。不含任何自环的图称作简单图 (simple graph), 也是本书主要讨论的对象。

通路 & 环路

所谓路径或通路 (path), 就是由 $m + 1$ 个顶点与 m 条边交替而成的一个序列:

$$\pi = \{ v_0, e_1, v_1, e_2, v_2, \dots, e_m, v_m \}$$

且对任何 $0 < i \leq m$ 都有 $e_i = (v_{i-1}, v_i)$ 。也就是说, 这些边依次地首尾相联。其中沿途的总数 m , 亦称作通路的长度, 记作 $|\pi| = m$ 。

为简化描述, 也可依次给出通路沿途的各个顶点, 而省略联接于其间的边, 即表示为:

$$\pi = \{ v_0, v_1, v_2, \dots, v_m \}$$

图6.2(a)中的 $\{C, A, B, A, D\}$ ，即是从顶点C到D的一条通路，其长度为4。可见，尽管通路上的边必须互异，但顶点却可能重复。沿途顶点互异的通路，称作简单通路（simple path）。在图6.2(b)中， $\{C, A, D, B\}$ 即是从顶点C到B的一条简单通路，其长度为3。

特别地，对于长度 $m \geq 1$ 的通路 π ，若起止顶点相同（即 $v_0 = v_m$ ），则称作环路（cycle），其长度也取作沿途边的总数。图6.3(a)中， $\{C, A, B, A, D, B, C\}$ 即是一条环路，其长度为6。反之，不含任何环路的有向图，称作有向无环图（directed acyclic graph, DAG）。

同样，尽管环路上的各边必须互异，但顶点却也可能重复。反之若沿途除 $v_0 = v_m$ 外所有顶点均互异，则称作简单环路（simple cycle）。例如，图6.3(b)中的 $\{C, A, B, C\}$ 即是一条简单环路，其长度为3。特别地，经过图中各边一次且恰好一次的环路，称作欧拉环路（Eulerian tour）——当然，其长度也恰好等于图中边的总数 e 。

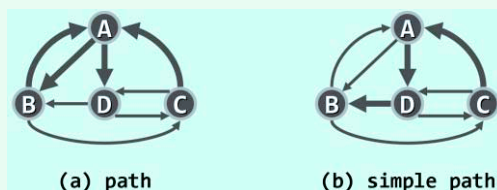


图6.2 通路与简单通路

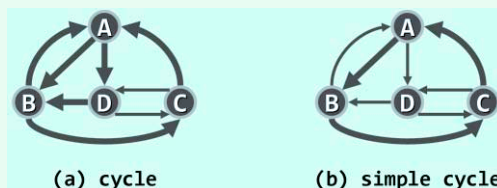


图6.3 环路与简单环路

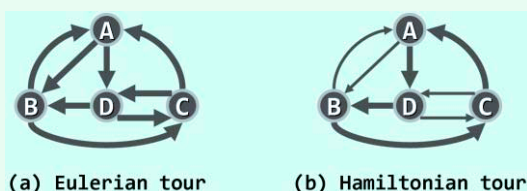


图6.4 欧拉环路与哈密顿环路

图6.4(a)中的 $\{C, A, B, A, D, C, D, B, C\}$ 即是一条欧拉环路，其长度为8。对偶地，经过图中各顶点一次且恰好一次的环路，称作哈密顿环路（Hamiltonian tour），其长度亦等于构成环路的边数。图6.4(b)中， $\{C, A, D, B, C\}$ 即是一条长度为4的哈密顿环路。

■ 带权网络

图不仅需要表示顶点之间是否存在某种关系，有时还需要表示这一关系的具体细节。以铁路运输为例，可以用顶点表示城市，用顶点之间的联边，表示对应的城市之间是否有客运铁路联接；同时，往往还需要记录各段铁路的长度、承运能力，以及运输成本等信息。

为适应这类应用要求，需通过一个权值函数，为每一边 e 指定一个权重（weight），比如 $wt(e)$ 即为边 e 的权重。各边均带有权重的图，称作带权图（weighted graph）或带权网络（weighted network），有时也简称网络（network），记作 $G(V, E, wt())$ 。

■ 复杂度

与其它算法一样，图算法也需要就时间性能和空间性能，进行分析和比较。相应地，问题的输入规模，也应该以顶点数与边数的总和（ $n + e$ ）来度量。不难看出，无论顶点多少，边数都有可能为0。那么反过来，在包含 n 个顶点的图中，至多可能包含多少条边呢？

对于无向图，每一对顶点至多贡献一条边，故总共不超过 $n(n - 1)/2$ 条边，且这个上界由完全图达到。对于有向图，每一对顶点都可能贡献（互逆的）两条边，因此至多可有 $n(n - 1)$ 条边。总而言之，必有 $e = O(n^2)$ 。

§ 6.2 抽象数据类型

6.2.1 操作接口

作为抽象数据类型，图支持的操作接口分为边和顶点两类，分列于表6.1和表6.2。

表6.1 图ADT支持的边操作接口

操 作 接 口	功 能 描 述
e()	边总数 E
exist(v, u)	判断联边(v, u)是否存在
insert(v, u)	引入从顶点v到u的联边
remove(v, u)	删除从顶点v到u的联边
status(v, u)	边(v, u)的状态
edge(v, u)	边(v, u)对应的数据域
weight(v, u)	边(v, u)的权重

表6.2 图ADT支持的顶点操作接口

操 作 接 口	功 能 描 述
n()	顶点总数 V
insert(v)	在顶点集V中插入新顶点v
remove(v)	将顶点v从顶点集中删除
inDegree(v) outDegree(v)	顶点v的入度、出度
firstNbr(v)	顶点v的首个邻接顶点
nextNbr(v, u)	在v的邻接顶点中，u的后继
status(v)	顶点v的状态
dTime(v)、fTime(v)	顶点v的时间标签
parent(v)	顶点v在遍历树中的父节点
priority(v)	顶点v在遍历树中的权重

6.2.2 Graph模板类

代码6.1以抽象模板类的形式，给出了图ADT的具体定义。

```
1 typedef enum { UNDISCOVERED, DISCOVERED, VISITED } VStatus; //顶点状态
2 typedef enum { UNDETERMINED, TREE, CROSS, FORWARD, BACKWARD } EStatus; //边状态
3
4 template <typename Tv, typename Te> //顶点类型、边类型
5 class Graph { //图Graph模板类
6 private:
7     void reset() { //所有顶点、边的辅助信息复位
8         for (int i = 0; i < n; i++) { //所有顶点的
9             status(i) = UNDISCOVERED; dTime(i) = fTime(i) = -1; //状态, 时间标签
10            parent(i) = -1; priority(i) = INT_MAX; // ( 在遍历树中的 ) 父节点, 优先级数
11            for (int j = 0; j < n; j++) //所有边的
12                if (exists(i, j)) status(i, j) = UNDETERMINED; //状态
13        }
14    }
15    void BFS(int, int&); // ( 连通域 ) 广度优先搜索算法
16    void DFS(int, int&); // ( 连通域 ) 深度优先搜索算法
17    void BCC(int, int&, Stack<int>&); // ( 连通域 ) 基于DFS的双连通分量分解算法
18    bool TSort(int, int&, Stack<Tv>*&); // ( 连通域 ) 基于DFS的拓扑排序算法
19    template <typename PU> void PFS(int, PU); // ( 连通域 ) 优先级搜索框架
```